VoltAgent / awesome-codex-subagents Public

Code Issues Pull requests 2 Actions Projects Security and quality Insights

main 2 Branches 0 Tags Go to file Go to file Code

necatiozmen Merge pull request #11 from haoxianhan/main 5f855c1 · last month

categories	add erlang expert	last month
CONTRIBUTING.md	add contributing guide	last month
LICENSE	update licence	last month
README.md	add erlang expert	last month

The awesome collection of 136+ Codex subagents across 10 categories.

awesome subagents 136 Last update march 111 online

More awesome collections for developers

Agent Skills 12k Claude Code Subagents 14k OpenClaw Skills 40k arxiv Agent Papers 328

Awesome Codex Subagents

This repository serves as the definitive collection of Codex Subagents, specialized AI assistants designed for specific development tasks. Writter specifically for Codex and aligned with the official docs.

README Contributing MIT license

Use Codex custom agent directories exactly as documented:

- `~/.codex/agents/` for global agents (available in all projects)
- `.codex/agents/` for project-specific agents (higher precedence in that repo)

1. Clone this repository.
2. Copy the `.toml` agent files you want into one of the directories above.

3. Restart or refresh your Codex session if needed.
4. Delegate explicitly in prompts (Codex does not auto-spawn custom subagents).

Examples:

```
mkdir -p ~/.codex/agents
cp categories/01-core-development/backend-developer.toml ~/.codex/agents/
```

```
mkdir -p .codex/agents
cp categories/04-quality-security/reviewer.toml .codex/agents/
```

If you use agent configuration in Codex, keep it in `.codex/config.toml` under `[agents]` as described in the official docs.

Subagent Storage Locations

Type	Path	Availability	Precedence
Project Subagents	`.codex/agents/`	Current project only	Higher
Global Subagents	`~/.codex/agents/`	All projects	Lower

Note: When naming conflicts occur, project-specific subagents override global ones.

Subagent Structure

Each subagent uses a Codex-native `.toml` format:

```
name = "subagent-name"
description = "When this agent should be invoked"
model = "gpt-5.3-codex-spark"
model_reasoning_effort = "medium"
sandbox_mode = "read-only"

[instructions]
text = """
You are a [role description and expertise areas]...

[Agent-specific checklists, patterns, and guidelines]...
"""
```

Smart Model Routing

Each subagent includes a `model` field that automatically routes it to the right model -- balancing quality and cost:

Model	When It's Used	Examples
`gpt-5.4`	Deep reasoning -- architecture reviews, security audits, financial logic	`security-auditor`, `architect-reviewer`, `fintech-engineer`
`gpt-5.3-codex-spark`	Fast scanning, synthesis, and lighter research tasks	`search-specialist`, `docs-researcher`, `agent-installer`

Sandbox Mode Philosophy

Each subagent's `sandbox_mode` field controls filesystem access:

- **Read-only agents** (reviewers, auditors): `sandbox_mode = "read-only"` - analyze without modifying
- **Workspace-write agents** (developers, engineers): `sandbox_mode = "workspace-write"` - create and modify files

Categories

01. Core Development

Essential development subagents for everyday coding tasks.

- api-designer - REST and GraphQL API architect
- backend-developer - Server-side expert for scalable APIs
- code-mapper - Code path mapping and ownership boundary analysis
- electron-pro - Desktop application expert
- frontend-developer - UI/UX specialist for React, Vue, and Angular
- fullstack-developer - End-to-end feature development
- graphql-architect - GraphQL schema and federation expert
- microservices-architect - Distributed systems designer
- mobile-developer - Cross-platform mobile specialist
- ui-designer - Visual design and interaction specialist
- ui-fixer - Smallest safe patch for reproduced UI issues
- websocket-engineer - Real-time communication specialist

02. Language Specialists

Language-specific experts with deep framework knowledge.

- angular-architect - Angular 15+ enterprise patterns expert
- cpp-pro - C++ performance expert
- csharp-developer - .NET ecosystem specialist
- django-developer - Django 4+ web development expert
- dotnet-core-expert - .NET 8 cross-platform specialist
- dotnet-framework-4.8-expert - .NET Framework legacy enterprise specialist
- elixir-expert - Elixir and OTP fault-tolerant systems expert
- erlang-expert - Erlang/OTP and rebar3 engineering expert
- flutter-expert - Flutter 3+ cross-platform mobile expert
- golang-pro - Go concurrency specialist
- java-architect - Enterprise Java expert
- javascript-pro - JavaScript development expert
- kotlin-specialist - Modern JVM language expert
- laravel-specialist - Laravel 10+ PHP framework expert
- nextjs-developer - Next.js 14+ full-stack specialist
- php-pro - PHP web development expert
- powershell-5.1-expert - Windows PowerShell 5.1 and full .NET Framework automation specialist
- powershell-7-expert - Cross-platform PowerShell 7+ automation and modern .NET specialist
- python-pro - Python ecosystem master
- rails-expert - Rails 8.1 rapid development expert
- react-specialist - React 18+ modern patterns expert
- rust-engineer - Systems programming expert
- spring-boot-engineer - Spring Boot 3+ microservices expert
- sql-pro - Database query expert
- swift-expert - iOS and macOS specialist
- typescript-pro - TypeScript specialist
- vue-expert - Vue 3 Composition API expert

03. Infrastructure

DevOps, cloud, and deployment specialists.

- azure-infra-engineer - Azure infrastructure and Az PowerShell automation expert
- cloud-architect - AWS/GCP/Azure specialist
- database-administrator - Database management expert
- deployment-engineer - Deployment automation specialist

- devops-engineer - CI/CD and automation expert
- devops-incident-responder - DevOps incident management
- docker-expert - Docker containerization and optimization expert
- incident-responder - System incident response expert
- kubernetes-specialist - Container orchestration master
- network-engineer - Network infrastructure specialist
- platform-engineer - Platform architecture expert
- security-engineer - Infrastructure security specialist
- sre-engineer - Site reliability engineering expert
- terraform-engineer - Infrastructure as Code expert
- terragrunt-expert - Terragrunt orchestration and DRY IaC specialist
- windows-infra-admin - Active Directory, DNS, DHCP, and GPO automation specialist

▶ **04. Quality & Security** — Testing, security, and code quality experts (16 agents)

▶ **05. Data & AI** — Data engineering, ML, and AI specialists (12 agents)

▶ **06. Developer Experience** — Tooling and developer productivity experts (13 agents)

▶ **07. Specialized Domains** — Domain-specific technology experts (12 agents)

▶ **08. Business & Product** — Product management and business analysis (11 agents)

▶ **09. Meta & Orchestration** — Agent coordination and meta-programming (12 agents)

▶ **10. Research & Analysis** — Research, search, and analysis specialists (7 agents)

Understanding Subagents

Subagents are specialized AI assistants that enhance Codex's capabilities by providing task-specific expertise. They act as dedicated helpers tha Codex can call upon when encountering particular types of work.

What Makes Subagents Special?

Independent Context Windows Every subagent operates within its own isolated context space, preventing cross-contamination between different tasks and maintaining clarity in the primary conversation thread.

Domain-Specific Intelligence Subagents come equipped with carefully crafted instructions tailored to their area of expertise, resulting in superior performance on specialized tasks.

Shared Across Projects After creating a subagent, you can utilize it throughout various projects and distribute it among team members to ensure consistent development practices.

Explicit Delegation Codex does not spawn subagents automatically. Use explicit delegation prompts to specify which agents to spawn, how to divide the work, and what shape the result should take.

Core Advantages

- **Memory Efficiency**: Isolated contexts prevent the main conversation from becoming cluttered with task-specific details
- **Enhanced Accuracy**: Specialized prompts and configurations lead to better results in specific domains
- **Workflow Consistency**: Team-wide subagent sharing ensures uniform approaches to common tasks
- **Codex-Native**: Uses `.toml` agent files aligned with official Codex subagent docs

Example Workflows

PR review workflow:

```
Review this branch with parallel subagents. Have reviewer look for correctness, security, and missing tests. Have
docs_researcher verify the framework APIs this patch depends on. Wait for both and summarize the findings with file
references.
```

Bug investigation workflow:

```
Investigate the broken settings flow. Have code_mapper trace the owning code paths, browser_debugger reproduce the bug in the
browser, and frontend_developer propose the smallest fix after the failure is understood. Wait for the read-heavy agents
first, then continue.
```

Repo exploration and planning workflow:

```
Use search_specialist to locate the code related to payment retries, knowledge_synthesizer to summarize the current design,
and refactoring_specialist to propose a minimal refactor plan. Return a concrete action list.
```

Contributing

We welcome contributions! See CONTRIBUTING.md for guidelines.

- Submit new subagents via PR
- Improve existing definitions
- Report issues and bugs

License

MIT License - see LICENSE

This repository is a curated collection of subagent definitions contributed by both the maintainers and the community. All subagents are provided "as is" without warranty. We do not audit or guarantee the security or correctness of any subagent. Review before use, the maintainer accept no liability for any issues arising from their use.

If you find an issue with a listed subagent or want your contribution removed, please open an issue in this repository and we'll address it promptly.

Releases

No releases published

Packages

No packages published

Contributors 2

necatiozmen Necati Özmen

haoxianhan haoxian